ЛАБОРАТОРНАЯ РАБОТА №7

Паттерны «Итератор», «Фабричный метод» и рефлексия в Java

по курсу
Объектно-ориентированное программирование

Группа 6204-010302D

Студент: С.0.Куропаткин.

Преподаватель: Борисов Дмитрий Сергеевич.

Задание 1: Реализация паттерна «Итератор»

Модификация интерфейса TabulatedFunction

```
public interface TabulatedFunction extends Function, Iterable<FunctionPoint>
```

Реализация итератора в ArrayTabulatedFunction

Добавлен метод iterator() с анонимным классом:

```
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int currentIndex = 0;

        @Override
        public boolean hasNext() {
            return currentIndex < size;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException();
            }
            return (FunctionPoint) points[currentIndex++].clone();
        }

        @Override
        public void remove() {
            throw new UnsupportedOperationException();
        }
    };
}
```

Реализация итератора в LinkedListTabulatedFunction

Создан эффективный итератор для связного списка:

```
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {

        private FunctionNode current = head.next;

        public boolean hasNext() {
            return current != head;
        }

        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException();
            }

            FunctionPoint point = (FunctionPoint) current.point.clone();
            current = current.next;
            return point;
        }

        public void remove() {
            throw new UnsupportedOperationException();
        }
    };
}
```

Задание 2
Реализация паттерна «Фабричный метод»

Создание интерфейса фабрики

```
package functions;

public interface TabulatedFunctionFactory {
    TabulatedFunction createTabulatedFunction(double leftX, double rightX,
int pointsCount);
    TabulatedFunction createTabulatedFunction(double leftX, double rightX,
double[] values);
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points);
}
```

Реализация фабрик для конкретных классов

```
public static class ArrayTabulatedFunctionFactory implements
TabulatedFunctionFactory {

    public TabulatedFunction createTabulatedFunction(double leftX, double
rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    public TabulatedFunction createTabulatedFunction(double leftX, double
rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points)
{
        return new ArrayTabulatedFunction(points);
    }
}
```

```
public static class LinkedListTabulatedFunctionFactory implements
TabulatedFunctionFactory {

    public TabulatedFunction createTabulatedFunction(double leftX, double
rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    public TabulatedFunction createTabulatedFunction(double leftX, double
rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points)
{
        return new LinkedListTabulatedFunction(points);
    }
}
```

Интеграция фабрики в TabulatedFunctions

```
private static TabulatedFunctionFactory factory = new
ArrayTabulatedFunction.ArrayTabulatedFunctionFactory();

// Приватный конструктор
private TabulatedFunctions() {}

// метод изменения фабрики
public static void setTabulatedFunctionFactory(TabulatedFunctionFactory
factory) {
    TabulatedFunctions.factory = factory;
}

// 3 метода createTabulatedFunction

public static TabulatedFunction createTabulatedFunction(FunctionPoint[]
points) {
    return factory.createTabulatedFunction(points);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double
rightX, int pointsCount) {
    return factory.createTabulatedFunction(leftX, rightX, pointsCount);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double
rightX, double[] values) {
    return factory.createTabulatedFunction(leftX, rightX, values);
}
```

Задание 3

Использование рефлексии

Добавление методов с рефлексией в TabulatedFunctions

```
public static TabulatedFunction createTabulatedFunction(Class<? extends
TabulatedFunction> functionClass, FunctionPoint[] points) {
    try {
        return functionClass
                .getConstructor(FunctionPoint[].class)
                .newInstance((Object) points);
    }
    catch (Exception e) {
        throw new IllegalArgumentException(e);
    }
}

public static TabulatedFunction createTabulatedFunction(Class<? extends
TabulatedFunction> functionClass, double leftX, double rightX, int
pointsCount) {
    try {
        return functionClass
                .getConstructor(double.class, double.class, int.class)
                .newInstance(leftX, rightX, pointsCount);
    }
    catch (Exception e) {
        throw new IllegalArgumentException(e);
    }
}
```

```
public static TabulatedFunction createTabulatedFunction(Class<? extends
TabulatedFunction> functionClass, double leftX, double rightX, double[]
values) {
    try {
        return functionClass
                .getConstructor(double.class, double.class, double[].class)
                .newInstance(leftX, rightX, values);
    }
    catch (Exception e) {
        throw new IllegalArgumentException(e);
    }
}
```

Перегрузка метода tabulate с рефлексией

```
public static TabulatedFunction tabulate(Class<? extends TabulatedFunction>
functionClass, Function function, double leftX, double rightX, int
pointsCount) throws InappropriateFunctionPointException {
    if (leftX < function.getLeftDomainBorder() - EPSILON ||
            rightX > function.getRightDomainBorder() + EPSILON)
    {
        throw new IllegalArgumentException();
    }

    FunctionPoint[] points = new FunctionPoint[pointsCount];
    double interval = Math.abs(rightX - leftX) / (pointsCount - 1);

    for (int i = 0; i < pointsCount; ++i) {
        double currentX = leftX + i * interval;

        if (currentX < function.getLeftDomainBorder() - EPSILON ||
                currentX > function.getRightDomainBorder() + EPSILON)
        {
            throw new IllegalArgumentException();
        }

        points[i] = new FunctionPoint(currentX,
function.getFunctionValue(currentX));
    }

    return createTabulatedFunction(functionClass, points);
}
```

Вывод Main:
Задание 1:
ArrayTabulatedFunction:
(-1.0; 0.36787944117144233)
(-0.5; 0.6065306597126334)
(0.0; 1.0)
(0.5; 1.6487212707001282)
(1.0; 2.718281828459045)

LinkedListTabulatedFunction:
(-1.0; 0.36787944117144233)

(-0.5; 0.6065306597126334)
(0.0; 1.0)
(0.5; 1.6487212707001282)
(1.0; 2.718281828459045)
Задание 2
class functions.ArrayTabulatedFunction
class functions.LinkedListTabulatedFunction
class functions.ArrayTabulatedFunction
Задание 3
class functions.ArrayTabulatedFunction
{(0.0; 0.0), (5.0; 0.0), (10.0; 0.0) }
class functions.ArrayTabulatedFunction
{(0.0; 0.0), (10.0; 10.0) }
class functions.LinkedListTabulatedFunction
{(0.0; 0.0), (10.0; 10.0) }
class functions.LinkedListTabulatedFunction
{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474), (0.6283185307179586; 0.5877852522924731), (0.9424777960769379; 0.8090169943749475), (1.256637061435917 2; 0.9510565162951535), (1.5707963267948966; 1.0), (1.8849555921538759; 0.9510565162951536), (2.199114857512855; 0.8090169943749475), (2.5132741228718345; 0.5877852522924732), (2.827433388230814; 0.3090169943749475), (3.141592653589793; 1.2246467991473532E-16) }